Building a GEO/AI Mentions Monitor with AI

A Step-by-Step Guide for Communications Professionals (or Other Total Beginners!)

What You're Building (AKA “Vibe-Coding”) Using AI

A dashboard that tracks how often your brand is mentioned when people ask AI models questions in your industry — and how that compares to competitors. Results are exportable as a shareable report for your team.

Before You Start: Four Decisions to Make First

These decisions will shape everything. Making them before you start will save you significant rework.

1. Lock down your query list. Ask your SEO team, if you have one, for common search terms by users. Finalize the exact questions you want to test before building anything. Changing them after the dashboard is built means your historical data becomes incomparable over time. If you’re looking to extrapolate your questions across multiple different cities (e.g. “Best daycare in [Minneapolis]/[St Paul]/[Minnesota]” aim for 6–10 city-specific query templates. (Fewer queries for more cities – you don’t want 10 queries for 10 cities, as this rapidly gets to 100 queries per AI model, which would take a long time.) You can also add national queries and/or non-geographical queries.

2. Decide which AI models you want to track. We recommend starting with Claude, Gemini, and ChatGPT. Add Perplexity once the others are stable. We chose not to add Co-Pilot or Grok. Each model requires a separate account, API key, and billing setup — factor in that time and do it first. Instructions for this are in Phase 1 below.

3. Identify your brand competitors upfront. The dashboard can check for competitor mentions in the same scan at no extra cost — it reads the same AI responses for multiple brand names simultaneously. Decide who you want to track before you brief your AI on the build.

4. Consider what your dashboard and resulting report should look like. Describe the finished product as if briefing a designer. Think about what tabs you want, what results charts, what the published report should contain, and whether you want it branded to your organisation. The more specific you are upfront, the less rework later.

Phase 1: Set Up Your Technical Environment

Install Python

The dashboard requires a small background program called a proxy server to work. Think of it as a middleman that sits on your computer and routes requests between your browser and the AI services — your browser has a security rule that prevents it from talking to external APIs directly, and the proxy bypasses this.

Windows: The proxy runs in Python, so you need Python installed first:

1. Go to http://python.org and download the latest version
2. Run the installer — make sure to tick **"Add to PATH"** during installation
3. You only need to do this once

For Mac: download from python.org as above, but on Mac you may get a security warning the first time you open the installer. Go to System Settings → Privacy & Security and click "Open Anyway"

Set Up Your API Accounts

Each AI provider has its own platform. Do this before building anything:

- **Claude:** http://console.anthropic.com
- **ChatGPT:** http://platform.openai.com
- **Gemini:** http://aistudio.google.com
- **Perplexity:** http://perplexity.ai/settings/api

For each one:

1. Create an account
2. Add billing — free tiers sound appealing but will fail mid-scan. Don't rely on them. You'll only need to add the minimum amount of credit (likely $5) to get started per AI model.
3. Generate an API key and save it somewhere safe
4. If you get stuck on any of this, ask your AI tool to walk you through the process

Lesson learned the hard way: Hours of apparent "technical bugs" in our own build turned out to be quota and billing issues. Confirm billing is working on every platform before you start building.

Phase 2: Brief Claude and Build the Dashboard

How to Structure Your Brief

Open a new Claude conversation and describe what you want in detail — not just "build me a GEO dashboard" but a full description of the finished product. A strong brief includes:

Functionality:

- Which AI models to scan
- Your brand name to detect in responses
- Your competitor names and any alternate spellings to watch for (e.g. "REW" and "REW.ca")
- Your city list and query templates

Layout — be specific about tabs and sections. For example:

- A tab to configure API keys and enable/disable models ("Models & Keys")
- A tab to manage your query list and cities ("Queries & Cities")
- A results view showing each query and whether your brand was mentioned
- A trend chart showing your mention rate over time
- A competitor share-of-voice chart for the last 12 scans

The published report:

- Explicitly ask for an export button that generates a standalone shareable HTML file
- Ask for filter buttons (by model, by city) if you want them
- Describe what sections the report should contain

Design:

- Provide your brand colours (hex codes) and font name if you want it on-brand
- Ask for a professional, polished design rather than a default browser look

What to Expect from the First Version

Your first version will be rough — that's normal. Treat this less like running a program and more like briefing a junior designer who is very capable but needs explicit direction.

The iteration process looks like this:

1. Claude produces a file — download it and review it in your browser
2. Identify what's wrong, missing, or different from what you imagined
3. Describe the changes to Claude specifically: not "make it better" but "add a tab called X that shows Y" or "change the chart to a grouped bar chart showing the last 12 scans"
4. Claude produces an updated file — download and replace the old one
5. Repeat until it matches your vision

Expect 3–4 sessions of iteration before it feels finished. Most of the time is spent on decisions and refinements, not on Claude's ability to build.

What We Built (As a Reference Point)

In this project, the dashboard ended up with four tabs:

- **Scan/Results** — the main view where you run scans, see live progress, and read each result
- **Trend History** — a bar chart of HouseSigma's mention rate over time, plus a grouped competitor chart
- **Queries & Cities** — where you manage your query templates and city list
- **Models & Keys** — where you paste API keys and enable/disable each model

The published report is a single, interactive HTML page that contains a scan overview with key metrics, a mention rate trend chart, a competitor share-of-voice chart, and the full list of query results, with filter buttons for model and city. The query results are expandable so you can see how your brand shows up (or doesn’t) in the wording of the result.

None of this appeared automatically — each section was explicitly requested and refined across multiple sessions.

Phase 3: Get the Proxy Running

Claude will produce two files: the dashboard HTML file and a proxy Python file. Both need to live *in the same folder* on your computer.

Each time you want to use the dashboard:

1. Open Command Prompt - type Command Prompt into the search bar, if using Windows
2. Type cd Downloads (or wherever your folder is) and press Enter
3. Type python geo-proxy.py and press Enter
4. Keep this window open — the proxy must stay running during scans
5. Open your browser and go to http://localhost:8765

Mac deviation: Open **Terminal** instead of Command Prompt (Applications → Utilities → Terminal, or Cmd + Space and type "Terminal"). Use python3 geo-proxy.py instead of python geo-proxy.py

Critical: Never open the dashboard by merely double-clicking the HTML file. Always use the proxy URL (http://localhost:8765). The file version will open, but it won't be able to make API calls to run a scan.

If at any time you update the dashboard file (because you and your AI made changes), you need to replace the file in your folder and restart the proxy before the new version will appear in your browser. To restart: press Ctrl+C in the Command Prompt (Mac: Terminal) window, then run the command again.

Phase 4: Verify AI Model Names Before You Build

AI providers update, rename, and retire their models frequently — sometimes with very little notice. A wrong model name produces a cryptic error that looks like a technical bug but is actually just an outdated string.

Always verify current model names from official documentation before building, not from memory or from what worked last month:

- **Anthropic:** docs.anthropic.com/en/docs/about-claude/models/overview
- **Google:** ai.google.dev/gemini-api/docs/models
- **OpenAI:** platform.openai.com/docs/models

- **Perplexity:** docs.perplexity.ai

Ask Claude to verify model names for you — it can search these pages in real time. But make this a specific step, not an afterthought.

Lesson learned: At the time of writing, the correct API strings were claude-sonnet-4-6, gemini-2.5-flash-lite, gpt-5.4, and Perplexity's sonar-pro. These will change frequently, and you will need to keep updating this and replacing the dashboard. Check before you build, and check each month when you run your scan.

Phase 5: Enter Your API Keys and Run Your First Scan

1. Go to **Models & Keys** in the dashboard
2. Paste in each API key and enable a model for the test scan
3. Click **Run Scan** with just one model enabled first
4. If it doesn't work, tell your AI what's happening and troubleshoot until it does
5. If it works (and it should!), enable all models and Run Scan again for the full results
6. While scanning, always keep the browser window visible/active and your computer awake — long scans can time out if the connection drops
7. With 3–4 models and 50 queries, expect the scan to take 10–20 minutes

Lesson learned: Run one model at a time first to confirm each is working before running all models together. It's much easier to debug one broken thing than three simultaneously.

One thing to know about your data: API keys and scan history are saved in your browser's local storage — they persist between sessions on the same computer and browser, but don't live inside the HTML file. This means:

- Replacing the HTML file with an updated version won't erase your history
- Clearing your browser data will erase your history
- Switching computers means re-entering your keys

Phase 6: Publish and Share Results

Once a scan completes, click **Publish Results** (or whatever you named your export button) to download a standalone HTML report. This file can be shared however you choose. Our process was:

- Commit the HTML file to a public GitHub repository and served via GitHub Pages
- Embedded in a Notion page using the GitHub Pages URL (a public URL that you can also share directly as a webpage
- Notion page with embedded report plus explainer/how-to-use text shared with team

The published report is a completely self-contained file — anyone can open it in a browser without needing the dashboard, the proxy, or any API keys.

Good luck!